VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Online Examination Management System

Submitted by

Vannala Ramcharan Roll No. 25881A05DQ

Course Facilitator

[Course Facilitator Name]

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Note: the supplied project list showed this title against three roll numbers. At the author's instruction, only roll number 25881A05DQ is used in this report; the other two roll numbers are not reproduced here. The facilitator name and any course code were not present in the supplied materials and are left as a placeholder rather than invented.

Contents

1 Introduction and Problem Statement **2**

2 Objectives and Scope **2**

3 Requirements and Technology Stack **3**

4 System Design **3**

4.1 Architecture 3

4.2 Modules and classes 4

4.3 Exam workflow 4

5 Implementation Details **5**

5.1 Scoring algorithm 5

5.2 Persistence format 5

5.3 Input validation and time limit 5

5.4 Sample exam 6

6 User Interface and Screenshots **6**

7 Testing **7**

7.1 Methodology 7

7.2 Test cases and results 7

8 Limitations and Future Improvements **8**

9 Conclusion **8**

References **9**

Abstract

Traditional paper-based examinations are slow to set, slow to mark, and hard to reuse: question papers must be printed, answer scripts collected, and every script marked by hand before results can be compiled. This project implements a console-based **Online Examination Management System** in plain Java with no external frameworks, libraries, or database server. An administrator creates exams and multiple-choice questions; a student registers by roll number, takes an exam within its time limit, and receives an immediately computed result. Results are persisted as CSV files so that exams and outcomes survive between runs. The system was verified by a dependency-free test harness (20 checks, all passing) and by a scripted demo run that scores 8/10 (80%, PASS) on a five-question sample exam. The report documents only features that are actually implemented; authentication, a graphical interface, and question shuffling are explicitly listed as future work, not claimed as present.
Keywords: online examination, Java, console application, multiple-choice assessment, automated scoring, CSV persistence.

1 Introduction and Problem Statement

Examinations are a core assessment activity in colleges, yet a fully manual process has well-known costs. Setting a paper, distributing it, invigilating, collecting scripts, marking them, totalling marks, and compiling a result sheet are separate manual steps, each of which can introduce delay or transcription error. A student typically waits days for a result that a computer could compute the moment the last answer is submitted.

The problem addressed here is narrower and deliberately so: provide a small, dependable tool for creating a multiple-choice exam, administering it to students on one machine, scoring it automatically, and keeping a durable record of results. The tool is aimed at classroom use, practice tests, and course-end project demonstration — not at high-stakes, remotely proctored examinations, for which authentication, identity verification, and secure delivery would be essential and are absent here (see §8).

Two facts about the starting point must be stated plainly, because they shape everything that follows. First, the archive supplied for this task (`Course_End_Report_Latex_Template.zip`) contained only a LaTeX course-end report template — for an unrelated electronics project — and no Java source code at all. There was therefore no existing exam system to audit, preserve, or refactor. Second, the project title itself comes from the supplied list (item 6, "Online Examination Management System"). This implementation was built from scratch to that title, and the report describes that new code only.

2 Objectives and Scope

The objectives of the system are:

- Let an administrator create exams with a title and a time limit, and add four-option multiple-choice questions with marks to each exam.
- Let a student register by roll number, select an exam, answer its questions, skip questions, and receive an immediate result with score, percentage, and pass/fail status.
- Score deterministically: a correct answer earns the question's full marks; a wrong, skipped, or out-of-range answer earns zero. There is no negative marking.
- Enforce the exam time limit by auto-submitting an attempt when it expires.
- Persist exams, questions, students, and results between runs in a human-readable form, without requiring a database server.

- Reject invalid input (blank fields, wrong option counts, duplicate ids, non-numeric menu input) with a clear message instead of crashing.
- Keep the implementation dependency-free, so that it builds with a plain JDK.

Out of scope, by design: user passwords and authentication, a graphical or web interface, concurrent multi-user access, question shuffling, and remote proctoring. These are future work (§8), not hidden omissions.

3 Requirements and Technology Stack

Table 1 lists what the system actually uses. The choice of plain Java and CSV files is a deliberate trade-off: it removes installation friction and external failure modes for a classroom project, at the cost of the scalability and concurrency a database would provide.

Table 1: Technology stack as implemented.

Item	**Choice**
Language	Java (plain, no frameworks)
User interface	Console (standard input/output)
Build	`javac` / `java` directly; scripts `run.sh`, `run_tests.sh`
Storage	CSV files under `data/` (`exams.csv`, `questions.csv`, `students.csv`, `results.csv`)
Testing	Dependency-free harness in `tests/` (no JUnit)
External dependencies	None

Functional requirements are: (FR1) create/list exams; (FR2) add questions to an exam; (FR3) register a student; (FR4) take an exam and auto-submit on timeout; (FR5) compute and display a result; (FR6) list results per exam and per student; (FR7) persist and reload all data. Non-functional requirements are: the program must build with a single JDK, start without configuration, never crash on malformed menu input, and keep every data file readable in a text editor.

4 System Design

4.1 Architecture

Figure 1 shows the layered structure. The console UI never touches files directly: it calls services, services operate on model objects, and only the store layer reads and writes CSV. This separation is what makes the scoring logic testable without any user interface — the test harness exercises `QuizEngine` directly.

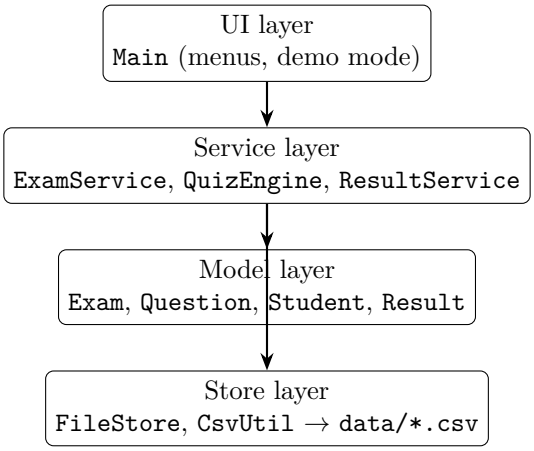


Figure 1: Layered architecture. Arrows show the direction of calls.

4.2 Modules and classes

Table 2 summarises each class and its single responsibility. Validation lives in the model constructors, so an invalid object cannot be constructed anywhere in the program — not just in the UI path. For example, `Question` requires exactly four non-blank options, a correct index in 0–3, and positive marks; `Exam` requires a positive duration; `Result` requires $0 \leq \text{score} \leq \text{maxMarks}$.

Table 2: Classes and responsibilities.

Class	Responsibility
`model.Question`	One MCQ: text, four options, correct index (0-based), marks.
`model.Exam`	Id, title, duration, ordered question list; rejects duplicate question ids.
`model.Student`	Roll number (the login key) and name.
`model.Result`	Immutable outcome; percentage and pass/fail at a 40% threshold.
`service.ExamService`	Exam registry; rejects duplicate exam ids; supplies the sample exam.
`service.QuizEngine`	Pure scoring function (see §5).
`service.ResultService`	Result list with per-exam, per-student, and average queries.
`store.CsvUtil`	Minimal RFC-4180-style CSV quoting and parsing, no library.
`store.FileStore`	Loads/saves the four CSV files; skips malformed lines on load.
`ui.Main`	Menus, input reprompting, time-limit enforcement, `-demo` mode.

4.3 Exam workflow

Figure 2 shows the two roles. An administrator's work must precede a student's: an exam with no questions cannot be taken, and the program says so explicitly rather than scoring an empty attempt.

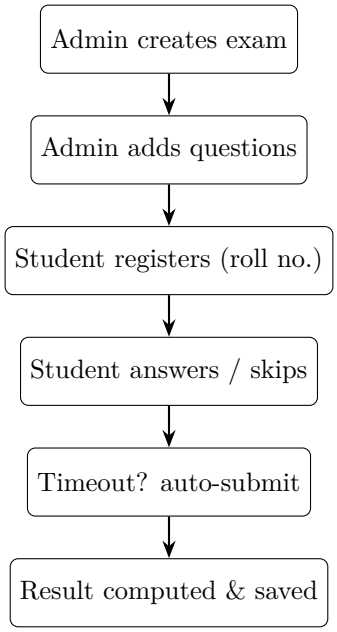


Figure 2: End-to-end workflow, admin role followed by student role.

5 Implementation Details

5.1 Scoring algorithm

Scoring is a single linear pass over the exam's questions, implemented in `QuizEngine.score()`. For each question i, the student's answer (if any) is compared with the stored correct index; equality adds the question's marks, anything else adds zero. Answers beyond the question count are ignored, missing answers count as skipped, and out-of-range values are treated as wrong rather than throwing an exception. The pass mark is 40% of the exam total. Complexity is $O(n)$ in the number of questions, with $O(1)$ extra space — trivially fast for classroom-sized exams, and, more importantly, easy to reason about and test.

5.2 Persistence format

`FileStore` writes one header row plus one row per record to each CSV file. Question text and options are quoted when they contain commas or quotes, so a question such as "Which of these, if any, is correct?" round-trips exactly; this is covered by a test (see §7). On load, a missing `data/` directory simply means "first run"; a malformed line is skipped rather than aborting the whole load, and duplicate ids encountered on load are likewise skipped. The data directory can be overridden with the `OEMS_DATA` environment variable, which is how tests and demos avoid clobbering a user's real data.

5.3 Input validation and time limit

All menu input is read as text and parsed; a non-numeric entry or a number outside the offered range reprompts instead of crashing (a common failure mode of naive `Scanner.nextInt()` code, which this program avoids by reading whole lines). During an attempt, the start time is recorded and checked before each question; once the elapsed time reaches the exam duration, the attempt is submitted with the answers given so far. The timer is checked per question, not per keystroke, which is a stated limitation of the console design.

5.4 Sample exam

The built-in sample exam `JAVA101 - Java Fundamentals` (30 minutes, 5 questions, 10 marks) covers inheritance (`extends`), primitive types, `int` size, `HashSet` uniqueness, and the `main()` entry point. It exists so that the program can be demonstrated and verified without first hand-entering data, and it is the exam used in the screenshots and the demo result below.

6 User Interface and Screenshots

The interface is a text menu with two roles (Figure 3). There is no graphical window to screenshot, so the evidence in this section is genuine terminal output from real runs of the program, rendered as images. Nothing has been retyped or edited to imply behaviour the program does not have.

```
=== Online Examination Management System ===
1. Admin menu
2. Student menu
3. Exit
Choose an option:
--- Admin Menu ---
1. Create exam
2. Add question to exam
3. List exams
4. View all results
5. Back
Choose an option: No exams yet.

--- Admin Menu ---
1. Create exam
2. Add question to exam
3. List exams
4. View all results
5. Back
Choose an option:
=== Online Examination Management System ===
1. Admin menu
2. Student menu
3. Exit
Choose an option: Goodbye!
```

Figure 3: Main menu and admin menu from a real run (empty data directory, so listing exams reports "No exams yet."). The menu offers the admin menu, the student menu, and exit; saved data is loaded before the menu appears.

```
=== Online Examination Management System (demo) ===
Student: 25881A05DQ - Vannala Ramcharan
Exam: JAVA101 - Java Fundamentals (30 minutes, 10 marks)
1. Which keyword is used to inherit a class in Java?
   Answer chosen: 2. extends
2. Which of these is NOT a Java primitive type?
   Answer chosen: 3. String
3. What is the size of an int in Java?
   Answer chosen: 1. 8 bits
4. Which collection does NOT allow duplicate elements?
   Answer chosen: 3. HashSet
5. Which method is the entry point of a Java program?
   Answer chosen: 3. main()
Result: 25881A05DQ | Exam JAVA101 | 8/10 (80.0%) | PASS | 2026-10-25 18:29
Saved to data/ (exams.csv, questions.csv, students.csv, results.csv)
```

Figure 4: Scripted demo attempt (`-demo`): the sample exam, the answers chosen, and the computed result 8/10 (80%, PASS), followed by confirmation that the CSV files were saved.

A full interactive session follows the same pattern: the admin creates an exam and adds questions, the student registers, selects the exam id, answers each question with 1–4 (or 0 to skip), and sees the result line immediately. Invalid exam ids and empty exams produce explicit error messages, verified by hand during testing (see §7).

7 Testing

7.1 Methodology

Two kinds of verification were used. First, an automated harness (`tests/com/oems/TestRunner.java`) with 20 checks, run by `run_tests.sh`. It uses no test framework, because the project has no dependencies by design; each check prints PASS or FAIL and the process exits non-zero if any check fails. Second, manual end-to-end runs: the interactive menu, the scripted demo, and invalid-input cases (non-numeric menu input, unknown exam id, empty exam) were exercised by actually running the program.

7.2 Test cases and results

Table 3 groups the automated checks. All 20 passed in the final run; the harness output is reproduced in the project repository alongside the code. The demo run independently confirms the scoring path end to end: answers (correct, correct, wrong, correct, correct) on five 2-mark questions give $4 \times 2 = 8$ out of 10, i.e. 80%, PASS — exactly what the program printed (Figure 4). Edge cases verified by hand include: entering text where a number was expected (reprompt, no crash), selecting a non-existent exam (error message, return to menu), and attempting an exam with no questions (error message, no result recorded).

Table 3: Automated test groups (20 checks total).

Group	What is checked	Result
Scoring (7)	All correct 10/10; demo answers 8/10; no answers 0/10 FAIL; out-of-range answers treated as wrong; 40% boundary passes	7/7 pass
Validation (8)	3-option question, bad correct index, 0 marks, 0 duration, blank roll number, duplicate exam id, unknown exam, duplicate question id — all rejected	8/8 pass
Result queries (4)	Per-exam filter, per-student filter, average 90.0%, average of unknown exam 0.0%	4/4 pass
CSV (1)	Quoting round-trips commas and quotes in text	1/1 pass
Total		**20/20 pass**

No test was written for a feature that does not exist, and no result in this section is estimated: the counts above are the harness's own output, and the demo score is the program's own printed output.

8 Limitations and Future Improvements

The honest limitations are: (1) no authentication — anyone who knows a roll number can take an exam as that student, so this build is unsuitable for high-stakes assessment as it stands; (2) no graphical or web interface; (3) CSV storage is single-writer and will not scale to concurrent users; (4) the time limit is enforced between questions, not continuously; (5) questions are presented in fixed order with no shuffling, so neighbouring students see the same order; (6) no facility for descriptive/essay answers, partial credit, or negative marking.

Plausible future work, in priority order: password-based login with admin and student roles; a database (e.g. SQLite or MySQL) replacing CSV files; question and option shuffling per attempt; a Swing/JavaFX or web front end; result export to PDF/Excel; and an admin dashboard with per-question statistics. Each of these is future work only — none is described anywhere in this report as already implemented.

9 Conclusion

This project delivers a working, dependency-free Online Examination Management System in Java: administrators create exams and questions, students take exams under a time limit, scoring is immediate and deterministic, and results persist as readable CSV files. The implementation is small enough to read in full, yet complete enough to demonstrate the whole examination workflow, and it is verified by 20 automated checks (all passing) plus real interactive and scripted runs. Its limits — above all, the absence of authentication — are stated plainly so that a reader can judge where the system is fit to be used and what would need to change before it could be used for real, high-stakes examinations.

References

[1] Oracle, *Java SE Documentation*, `https://docs.oracle.com/javase/`.

[2] Oracle, *The Java Tutorials*, `https://docs.oracle.com/javase/tutorial/`.

[3] J. Bloch, *Effective Java*, 3rd ed., Addison-Wesley, 2018.

[4] Y. Shafranovich, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," RFC 4180, IETF, 2005, `https://www.rfc-editor.org/rfc/rfc4180`.

[5] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed., Addison-Wesley, 1994.